FQRTT: Stateful Round Trip Time Estimation for Wireless Embedded Systems Using Q-Learning and Fuzzy Logic

CSE-322 Computer Networks Sessional Project

Dibya Jyoti Sarkar

Student ID: 2105084

1 Introduction

Round Trip Time (RTT) estimation has always been a hot research topic in the field of computer networks. Many methods have been applied by Jacobson [2], Karn and others. In this paper, authors have implemented a new method to estimate RTT which is Q-learning.

2 High Overview of Original Paper

The paper proposes QRTT [1], a novel RTT estimation mechanism for TCP that replaces the traditional Jacobson [2]/Karn estimator with a Q-learning–inspired, state-aware estimator. Instead of solely relying on successful RTT samples, QRTT [1] also relies on failure ones too. It models RTT as learning process of two states: successful or failure. Each state has it's own RTT estimate. The reason failure samples are also being considered because it enables faster adaptation to dynamic networks such as wireless or high jitter networks. Authors have used ns-2 for simulation. In simulation, it has been observed that TCP throughput and average energy per bit have improved more compared to Jacobson's algorithm [2] due to QRTT's intelligent RTT estimation [1].

QRTT [1] uses a slightly modified formula for RTT estimation:

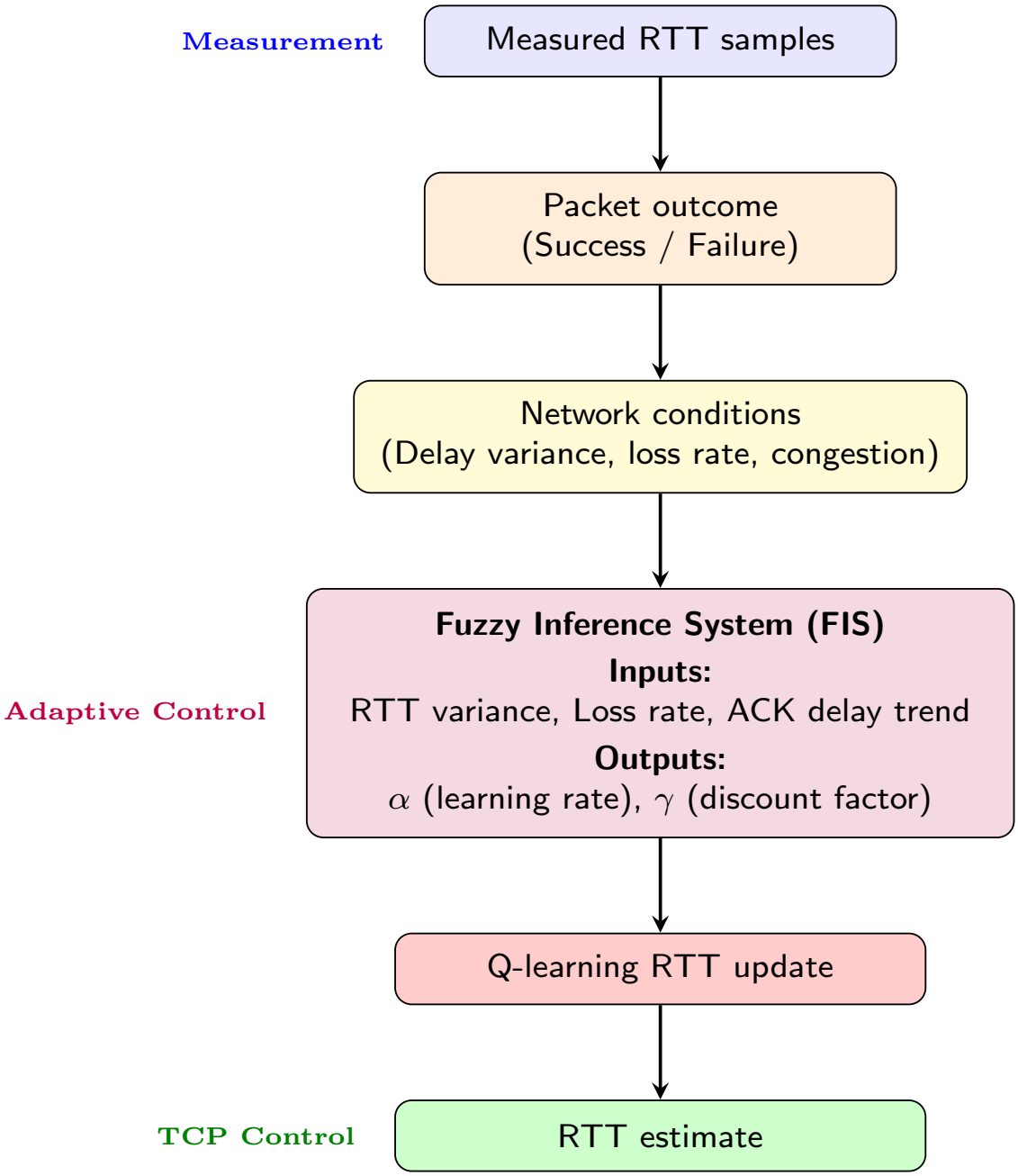
$$Q(s_t) \leftarrow Q(s_t) + \alpha_{s_t} \left(d_{t+1} + \gamma_{s_{t+1}} Q(s_{t+1}) - Q(s_t) \right)$$

where,

- $Q(s_t)$ denotes the estimated Round-Trip Time (RTT) value associated with the previous state s_t.
- s_t represents the current transmission state at time t, where the state can be either a successful transmission or a failure (timeout).

- α_{s_t} is the learning rate.
- d_{t+1} is the measured delay observed at time $t+1$.
- $\gamma_{s_{t+1}}$ is the discount factor associated with the next state s_{t+1}.
- $Q(s_{t+1})$ denotes the RTT estimate of the current state s_{t+1}.

3 Proposed Modifications with Architecture



We propose "Fuzzy Logic", an Artificial Intelligence technique to learn the "learning factor" and "discount factor". Instead of empirical tuning, we will use this method to tune the factors as they are the keys to improve QRTT's performance [1]

4 Network Topologies Under Simulation

The simulation were run for below topologies:

- Wired
- 802.15.4 (mobile)

5 Parameters Under Variation

The following simulation parameter combinations were considered:

- Low load:
  - Nodes = 20, Flows = 10, Packets per second = 100, Speed = 5.0
  - Nodes = 20, Flows = 10, Packets per second = 100, Speed = 25.0
  - Nodes = 20, Flows = 10, Packets per second = 500, Speed = 5.0
- Medium nodes with varying flows and packet rates:
  - Nodes = 60, Flows = 10, Packets per second = 100, Speed = 15.0
  - Nodes = 60, Flows = 20, Packets per second = 200, Speed = 10.0
  - Nodes = 60, Flows = 30, Packets per second = 300, Speed = 15.0
  - Nodes = 60, Flows = 40, Packets per second = 400, Speed = 20.0
  - Nodes = 60, Flows = 50, Packets per second = 500, Speed = 25.0
- High load:
  - Nodes = 100, Flows = 50, Packets per second = 500, Speed = 5.0
  - Nodes = 100, Flows = 50, Packets per second = 500, Speed = 25.0
  - Nodes = 100, Flows = 10, Packets per second = 100, Speed = 25.0
- Diagonal sweep where all parameters scale together:
  - Nodes = 20, Flows = 10, Packets per second = 100, Speed = 5.0
  - Nodes = 40, Flows = 20, Packets per second = 200, Speed = 10.0
  - Nodes = 60, Flows = 30, Packets per second = 300, Speed = 15.0
  - Nodes = 80, Flows = 40, Packets per second = 400, Speed = 20.0

6 Modifications Made in the Simulator

In the base paper, α (learning rate) and γ (discount factor) were not adaptive to the network environment. They were hardcoded for certain topologies. Our modification is to make α and γ adaptive to the network using the artificial intelligence technique known as fuzzy logic. The pseudocode is given below.

Algorithm 1: Fuzzy Adaptation of α and γ

Input: recent RTT samples, ACK delays, transmission events, timeout events
Output: updated α, γ

▷ Compute network indicators

$\sigma \leftarrow$ RTT standard deviation
$L \leftarrow$ loss rate
$T \leftarrow$ ACK delay trend

▷ Normalize indicators

$\sigma_n \leftarrow \text{NormalizeVariance}(\sigma)$
$L_n \leftarrow \text{NormalizeLoss}(L)$
$T_n \leftarrow \text{NormalizeTrend}(T)$

▷ Fuzzy memberships

$\mu_\sigma^{low} \leftarrow \text{Descending}(\sigma_n)$
$\mu_\sigma^{med} \leftarrow \text{Triangular}(\sigma_n)$
$\mu_\sigma^{high} \leftarrow \text{Ascending}(\sigma_n)$
$\mu_T^{fall} \leftarrow \text{Descending}(T_n)$
$\mu_T^{stable} \leftarrow \text{Triangular}(T_n)$
$\mu_T^{rise} \leftarrow \text{Ascending}(T_n)$
$\mu_L^{low} \leftarrow \text{Descending}(L_n)$
$\mu_L^{med} \leftarrow \text{Triangular}(L_n)$
$\mu_L^{high} \leftarrow \text{Ascending}(L_n)$

▷ Infer α using fuzzy rules

$r_1 \leftarrow \mu_\sigma^{high}$
$r_2 \leftarrow \mu_T^{rise}$
$r_3 \leftarrow \min(\mu_\sigma^{med}, \mu_T^{rise})$
$r_4 \leftarrow \min(\mu_\sigma^{med}, \mu_T^{stable})$
$r_5 \leftarrow \min(\mu_\sigma^{low}, \mu_T^{stable})$
$r_6 \leftarrow \min(\mu_\sigma^{low}, \mu_T^{fall})$
$\alpha \leftarrow \dfrac{r_1\alpha_{high} + r_2\alpha_{high} + r_3\alpha_{high} + r_4\alpha_{med} + r_5\alpha_{low} + r_6\alpha_{low}}{r_1 + r_2 + r_3 + r_4 + r_5 + r_6}$

▷ Infer γ using fuzzy rules

Define γ_{small}, γ_{medium}, and γ_{large} based on L_n
$r_1 \leftarrow \mu_L^{high}$
$r_2 \leftarrow \min(\mu_L^{med}, \mu_T^{rise})$
$r_3 \leftarrow \min(\mu_\sigma^{high}, \mu_L^{med})$
$r_4 \leftarrow \min(\mu_L^{low}, \mu_\sigma^{low}, \mu_T^{stable})$
$r_5 \leftarrow \min(\mu_L^{low}, \mu_\sigma^{med})$
$r_6 \leftarrow \min(\mu_L^{low}, \mu_T^{fall})$
$\gamma \leftarrow \dfrac{r_1\gamma_{small} + r_2\gamma_{small} + r_3\gamma_{small} + r_4\gamma_{large} + r_5\gamma_{medium} + r_6\gamma_{large}}{r_1 + r_2 + r_3 + r_4 + r_5 + r_6}$

▷ Clamp outputs

33: $\alpha \leftarrow \min(0.45, \max(0.05, \alpha))$
34: $\gamma \leftarrow \min(0.25, \max(-0.5, \gamma))$
35: **return** α, γ

7 Results with graphs

7.1 Wired Topology

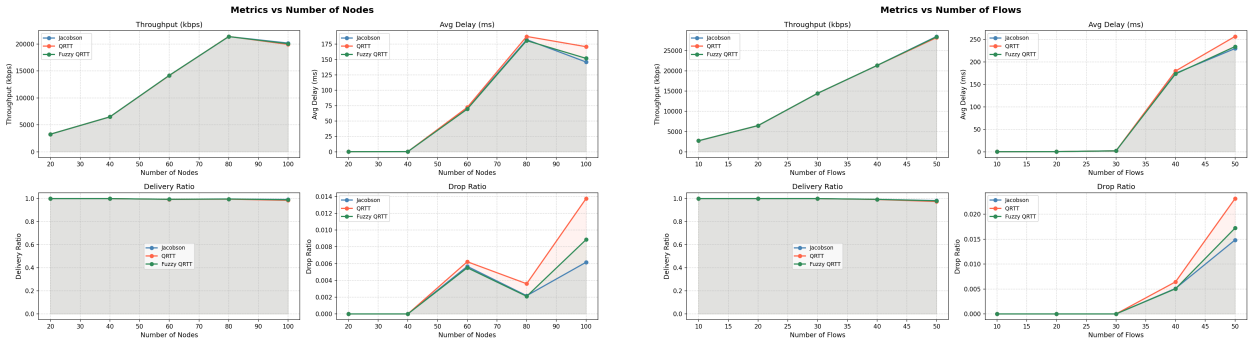


Figure 1: Wired topology results versus number of nodes and number of flows.

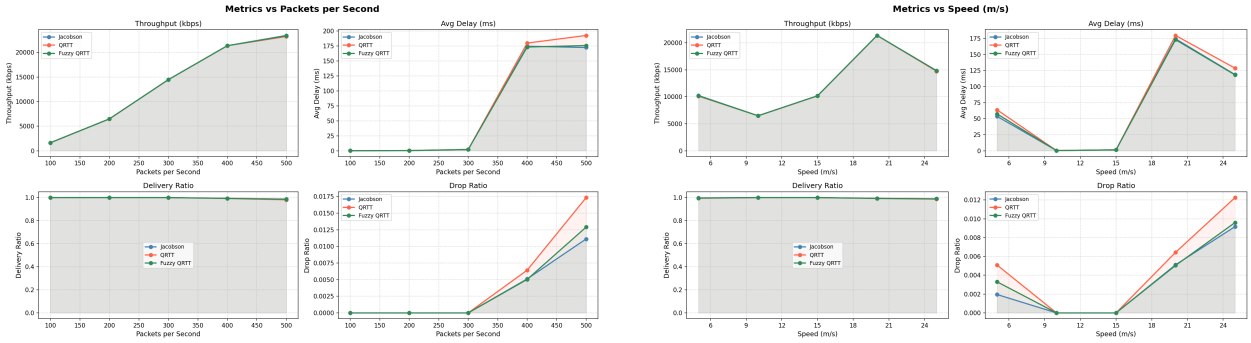


Figure 2: Wired topology results versus packets per second and node speed.

7.2 Wireless Topology

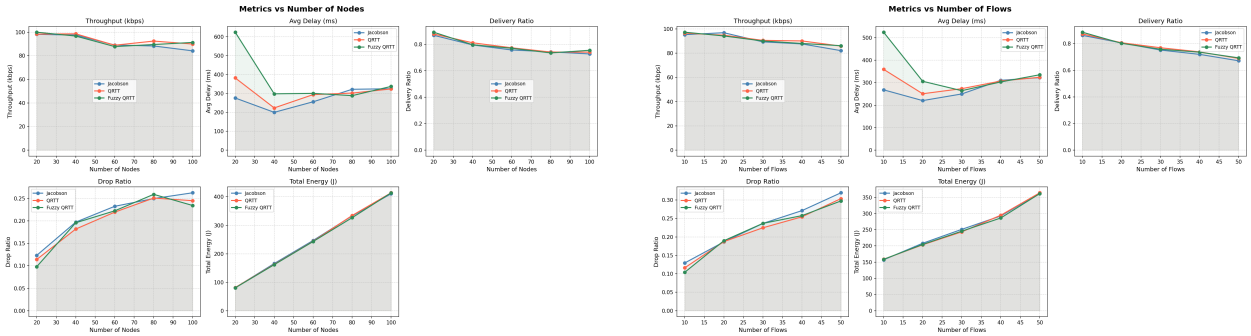


Figure 3: Wireless topology results versus number of nodes and number of flows.

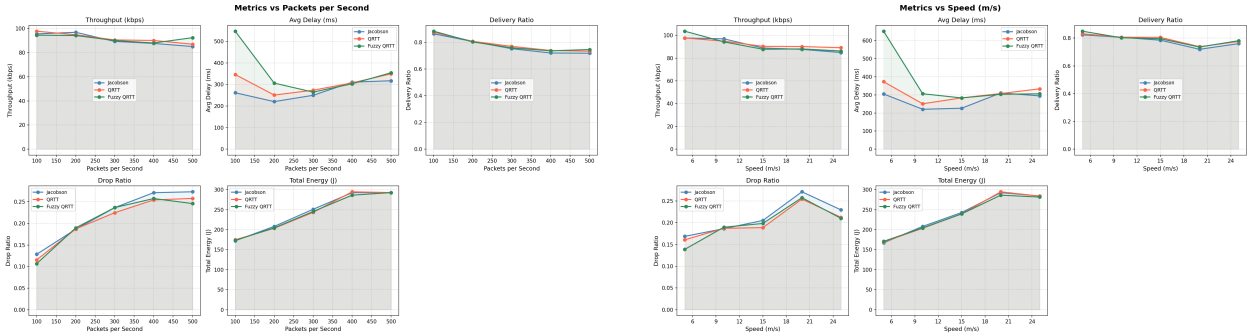
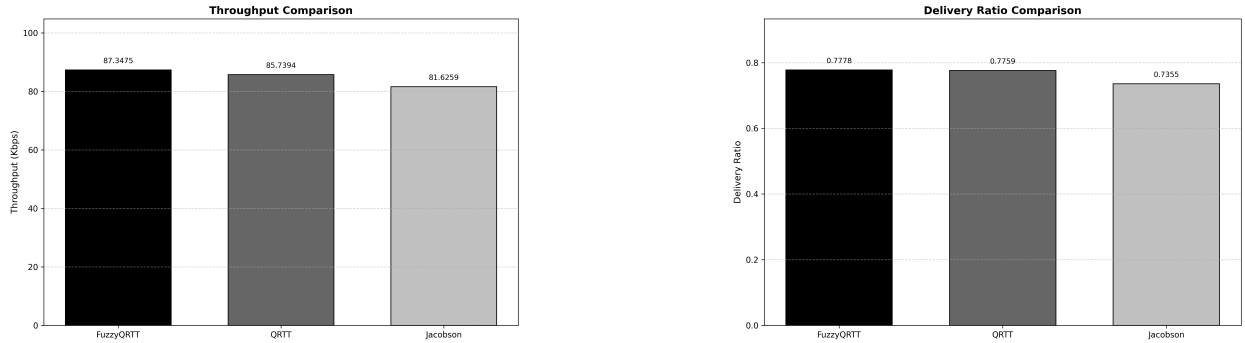


Figure 4: Wireless topology results versus packets per second and node speed.

8 Base paper graph with comparison



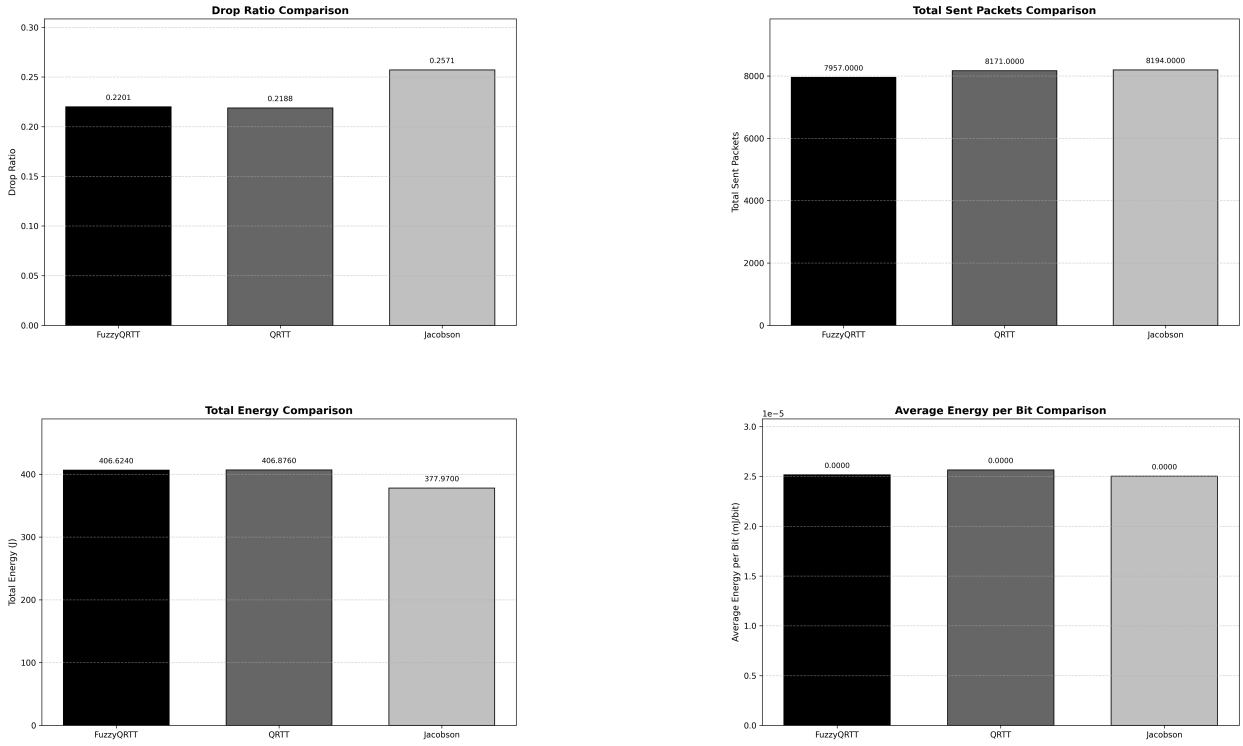


Figure: Comparison of throughput, delivery ratio, drop ratio, sent packets, total energy, and average energy per bit with the base paper.

9 Summary Findings

9.1 Base Paper

For base paper, we used below topology:

- Radio 802.15.4 (static)
- Coverage Area - 300 x 300 square meters
- No. of Nodes - 10 x 10
- Node Placement - Uniform
- No. of flows - 20
- Characteristic of flows - fixed cross flow

In the base paper, QRTT [1] showed improvement compared to the Jacobson algorithm [2]. This is because Jacobson only took account of success states, whereas QRTT took account of both failure

and success states.
However, the learning rate (α) and discount factor (γ) were topology specific. As γ_success was too harsh for the topology, retransmissions were occuring frequently. As a result, Total Energy was higher than the Jacobson algorithm [2].
In our FuzzyQRTT (FQRTT), we use fuzzy logic to make α and γ adaptive to network. As a result, we can see that there is a slight improvement compared to QRTT [1]. Also, total energy is less than QRTT as the γ was adaptive instead of hardcoded.

9.2 Wireless (Radio 802.15.4 mobile) Topology

In terms of parameters like flows, number of nodes, packet per second, speed, FQRTT's performance is slightly better than QRTT [1] and Jacobson [2].However, when speed increases, QRTT's throughput is better than FQRTT.
QRTT [1] also performs better than Jacobson [2], but sometimes Jacobson does better in terms of Average Delay (e.g., when packet per second increases) than QRTT,FQRTT. The most probable reason could be that Jacobson focuses on successful transmission where as QRTT and FQRTT have to focus on both success and failure states.

9.3 Wired Topology

In wired topology, FQRTT has slight improvement in throughputs with respect to flows, number of node, packets per second. However, its throughput performance is the same as QRTT [1] and Jacobson [2] with respect to speed.
Jacobson's [2] average delay and drop ratio are much better than FQRTT and QRTT [1]. It is expected because in wired networks, almost all losses are congestion-based. As a result, there is no link failure state to speak of. So splitting into two Q-values models a distinction that does not exist. It gives two noisy estimates instead of one stable one. Again, Jacobson [2] was designed for wired networks, so its RTT estimation is suitable for this type of network.
However, delivery ratio almost stays same for every algorithm.

10 Discussion

In this project, we implemented the base paper, which uses QRTT [1] for RTT estimation. It performs better than the Jacobson algorithm [2] in many metrics like throughput, total energy, delivery ratio, and drop ratio in wireless networks. Then we proposed FQRTT which uses fuzzy logic to learn α and γ through network parameters like RTT Variance, Loss Rate, Ack Delay.
FQRTT performed slightly better than QRTT [1] and Jacobson [2] in those metrics in wireless networks. However, in wired networks, the Jacobson algorithm [2] still outperformed in factors like drop ratio and average delay.

11 References

References

[1] A. B. M. Alim Al Islam and Vijay Raghunathan, *QRTT: Stateful Round Trip Time Estimation for Wireless Embedded Systems Using Q-Learning.*

[2] V. Jacobson, "Congestion avoidance and control," *ACM Computer Communication Review*, vol. 18, pp. 314–329, 1988.